

Rapport Intelligence Artificielle – Groupe 4

Beaussier Mélissa, Leportier Elliott, Prétot Cécilia

Table des matières

Introduction	1
Besoin client 1 : Visualisation sur carte	2
Préparation des données	2
Méthode de clustering n°1 : K-means	2
Méthode de clustering n°2 : MinibatchK-means	3
Méthode de clustering n°3 : Birch	4
Comparaison	5
Script	5
Besoin client 2 : Modèle de prédiction du type de navire.....	6
Préparation des données	6
Apprentissage supervisé pour la classification.....	7
Métriques pour la classification	7
Script	8
Besoin client 3 : Prédiction de la trajectoire des navires	9
Régression linéaire	9
Random forest.....	9
Régression polynomiale.....	10
Script	10
ANNEXE	11
Bibliographie	11

Introduction

Dans le cadre de la Semaine de l'Intelligence Artificielle en troisième année, ce projet a pour objectif de mobiliser de manière intégrée les compétences acquises en Big Data, Intelligence Artificielle, Développement Web et Bases de Données. Il s'articule autour d'un cas concret : l'analyse des comportements de navigation de navires à partir de données AIS (Automatic Identification System). L'enjeu est de développer une application complète capable de traiter, modéliser et visualiser ces données à des fins d'analyse, de classification et de prédiction, tout en respectant un cahier des charges technique et fonctionnel structuré autour de trois grands besoins clients.

Besoin client 1 : Visualisation sur carte

Le premier besoin du client vise à fournir une visualisation claire et intuitive des trajectoires maritimes. L'objectif est de regrouper automatiquement les navires selon des comportements de navigation similaires (vitesse, cap, heading...), afin d'identifier des profils typiques, détecter d'éventuelles anomalies ou encore optimiser les routes maritimes. Cela nécessite l'utilisation de techniques d'apprentissage non supervisé, notamment le clustering.

Préparation des données

Les données utilisées sont celles du fichier CSV `vessel_data_clean_v2.csv`, qui est le fichier csv contenant les données originelles nettoyées pendant la semaine de projet de Big Data.

Nous avons ensuite sélectionné les colonnes pertinentes au contexte. Ici, nous souhaitons étudier les comportements liés aux schémas de navigations. Plusieurs variables ont été présélectionnées dont : SOG, COG, Heading, LAT et LON. En effet, ces 5 variables sont liées aux

schémas de navigation des navires, mais il reste à vérifier qu'elles ne sont pas corrélées entre elles, ce qui pourrait fausser nos futurs résultats. Pour cela, on réalise la matrice de corrélation et on obtient ceci :

Matrice de corrélation :

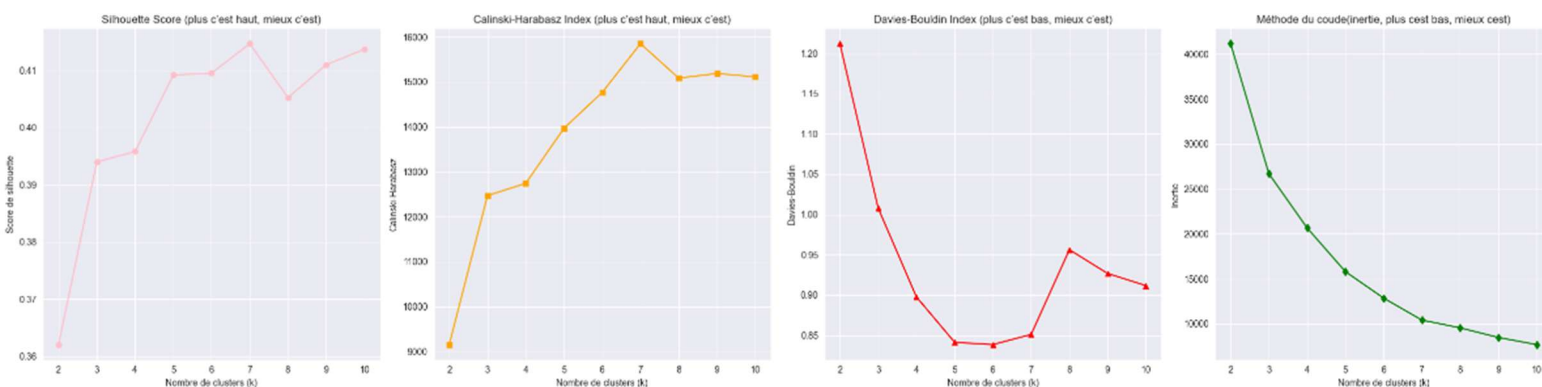
	SOG	COG	Heading	LAT	LON
SOG	1.000000	-0.105224	-0.189789	-0.224336	0.119182
COG	-0.105224	1.000000	0.282213	0.034988	-0.003799
Heading	-0.189789	0.282213	1.000000	0.142843	0.055488
LAT	-0.224336	0.034988	0.142843	1.000000	-0.578671
LON	0.119182	-0.003799	0.055488	-0.578671	1.000000

Comme on peut le voir, SOG, COG et Heading ne sont pas corrélées entre elles, mais LAT et LON sont trop corrélées avec les autres. On les retire donc de la sélection pour finir avec 3 variables : SOG, LON et Heading.

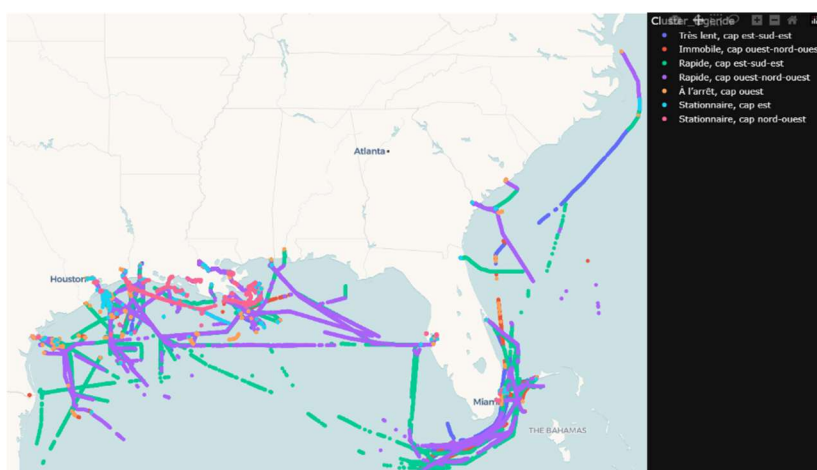
Méthode de clustering n°1 : K-means

La méthode K-means est un algorithme de clustering non supervisé qui vise à partitionner un ensemble de données en k groupes homogènes. Elle fonctionne en attribuant chaque point au centre de cluster le plus proche, puis en mettant à jour les centres jusqu'à convergence. K-means est simple à implémenter et efficace pour des jeux de données de taille modérée, mais sa sensibilité aux valeurs aberrantes et aux formes de clusters non sphériques peut limiter ses performances dans certains cas. Ici, nos données ont été nettoyées au préalable et les valeurs aberrantes ne sont plus un problème.

Il a d'abord fallu déterminer le nombre de cluster k le plus optimal. Nous avons réalisé plusieurs tests grâce : au Silhouette Coefficient, au Calinski-Harabasz Index, au Davies-Bouldin Index et la méthode du coude. Pour cela, nous avons testé la méthode K-means sur un échantillon de 20 000 données, et nous allons calculer ces 4 indicateurs. Chaque graphique représentera un indicateur selon le nombre « k » clusters choisi, à chaque fois moyennée avec un `random_state` allant de 0 à 4. Voilà les graphiques réalisés :



Pour le coefficient de Silhouette, on veut obtenir le plus haut chiffre possible (car plus il est élevé, plus les clusters sont compacts et bien séparés). De même pour l'index de Calinski-Harabasz (car plus il est élevé, meilleure est la compacité des clusters), et pour l'index de Davies-Bouldin on souhaite le plus bas chiffre possible (car moins il est élevé, moins les clusters sont similaires entre eux). Pour la méthode du coude, on cherche les cassures. Après observation des graphiques, on choisira $k = 7$ pour le nombre de clusters à appliquer à la méthode K-mean.

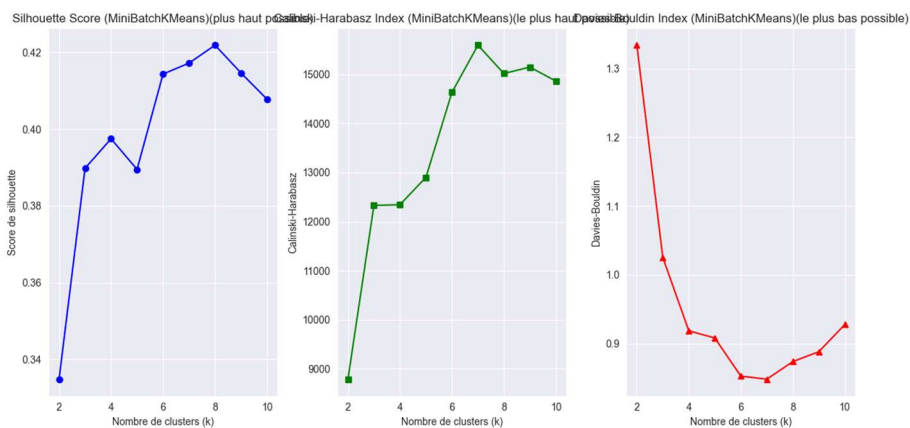


Après avoir appliqué $k=7$ à la méthode K-mean, il ne reste plus qu'à appliquer la méthode. Pour obtenir le clustering :

Méthode de clustering n°2 : MinibatchK-means

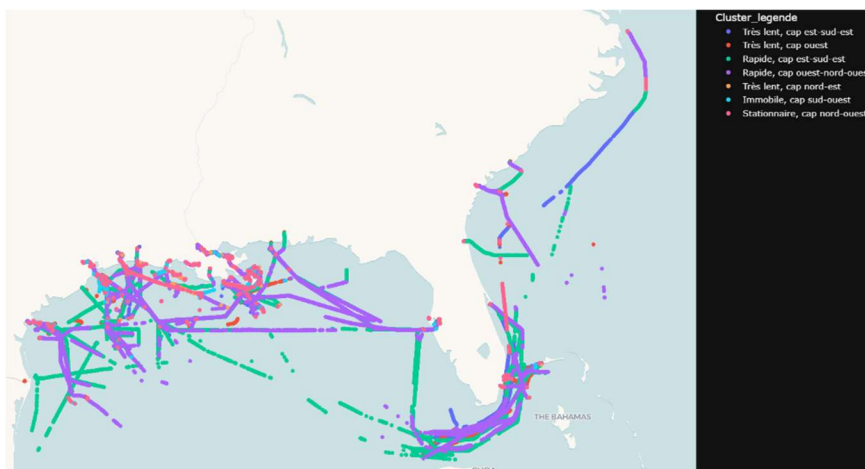
MiniBatchKMeans est une variante plus rapide de K-means, conçue pour traiter efficacement de grands ensembles de données. Au lieu d'utiliser l'ensemble complet des données à chaque itération, l'algorithme utilise de petits sous-échantillons (mini-batches) pour mettre à jour les centres. Cela permet une convergence plus rapide avec un coût computationnel réduit, tout en obtenant des résultats comparables à ceux de K-means dans de nombreux cas.

Il a encore une fois fallu trouver le nombre de clusters k le plus optimal :



Après observation des graphiques, on choisira k=7 pour le nombre de clusters à appliquer à la méthode MiniBatchK-mean.

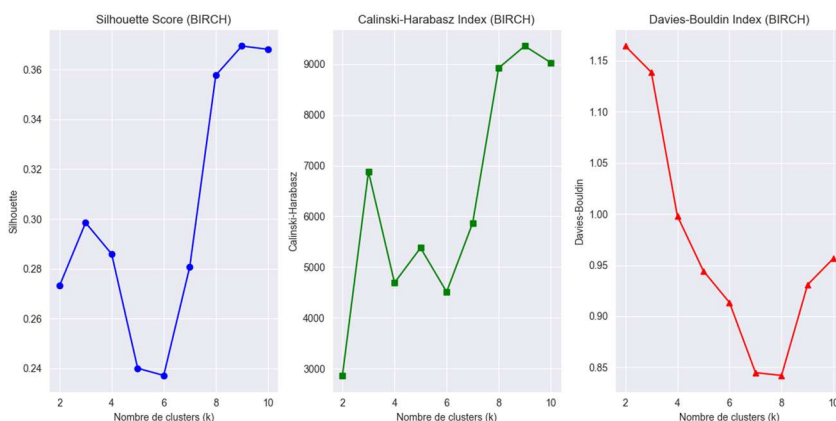
Après avoir appliqué k = 7 à la méthode MiniBatchK-mean, il ne reste plus qu'à appliquer la méthode. Pour obtenir le clustering :



On remarque que mise à part les couleurs qui sont différentes, le clustering à l'air plutôt identique. Cela était plutôt attendu étant donné que MiniBatchK-mean est une variation de la méthode K-mean, adaptée et optimisant pour une plus grande quantité de données, mais donnant des résultats similaires.

Méthode de clustering n°3 : Birch

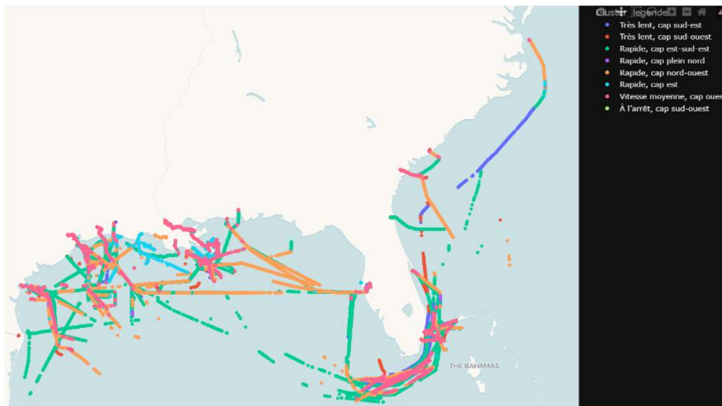
Birch est un algorithme de clustering hiérarchique particulièrement adapté aux très grands ensembles de données. Il construit progressivement une structure arborescente qui résume les



données sous forme de sous-clusters, avant de leur appliquer éventuellement une méthode de regroupement finale. Birch est efficace pour les données continues et permet une bonne gestion de la mémoire, mais peut être moins performant lorsque les données présentent des formes de clusters très irrégulières. Nous avons également évalué nos 3

indicateurs afin de choisir le meilleur nombre de clusters :

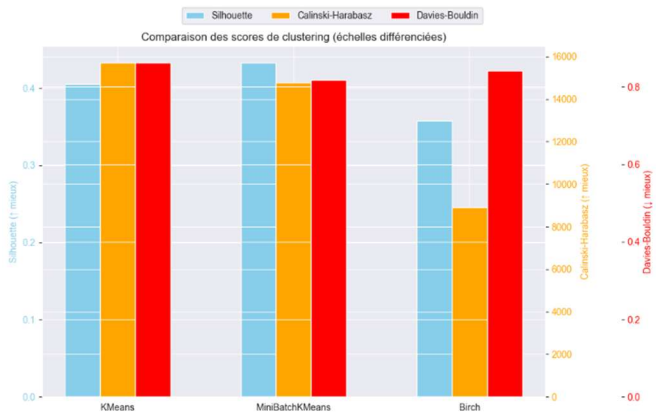
Après observation, on choisira un nombre de clusters $k = 8$ pour appliquer la méthode Birch.



Après avoir appliqué $k = 8$ à la méthode Birch, il ne reste plus qu'à appliquer la méthode. Pour obtenir le clustering :

Etant donné que nous avons un cluster de plus, le clustering est un peu différent mais reste dans l'ensemble assez identique.

Comparaison



MiniBatchKMeans semble offrir le meilleur compromis global en termes de compacité et de séparation des clusters. KMeans est très proche et légèrement meilleur sur Calinski-Harabasz, donc pertinent si l'on préfère optimiser la séparation stricte.

Si la vitesse d'entraînement devient notre critère numéro 1, ce qui est plutôt le cas ici étant donné le grand nombre de données et la puissance de nos ordinateurs, MiniBatchKMeans est à privilégier.

	Silhouette Coefficient	Calinski-Harabasz Index	Davies-Bouldin Index
K-means (k = 7)	0.3577	15697.95	0.8421
MiniBatchK-means (k = 7)	0.4324	14768.50	0.8186
Birch (k = 8)	0.4052	8922.62	0.8618

Script

Le script final consiste en la console qui demande à l'utilisateur les valeurs de SOG, COG et Heading d'un navire. Ces données sont passées dans le modèle scaler.pkl et model_minibatch.pkl afin d'afficher le cluster correspondant au navire.

```
Entrez la vitesse de votre navire:12.4
Entrez la cap vrai de votre navire:78.3
Entrez le heading de votre navire:80
Cluster (MiniBatchKMeans): 3

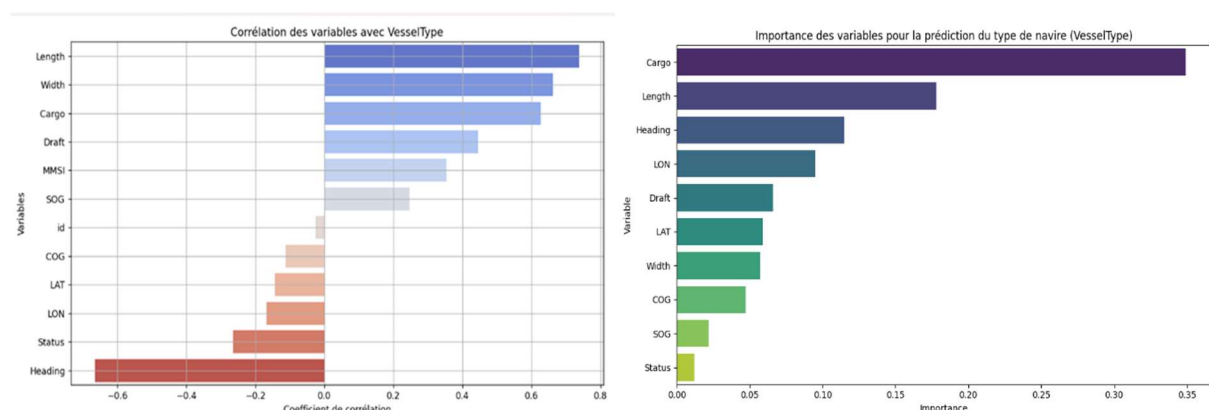
Process finished with exit code 0
```

Besoin client 2 : Modèle de prédiction du type de navire

Notre deuxième Client souhaitait développer un modèle d'intelligence artificielle basé sur un apprentissage supervisé, c'est-à-dire un modèle qui se base sur une base de données. La base de données utilisée contenait des informations sur des navires que nous avons préalablement filtrés. Notre modèle doit être capable de prédire le type d'un navire en fonction de ses caractéristiques. Pour cela, il faut réaliser plusieurs étapes : la préparation des données, le choix d'un modèle d'apprentissage et des caractéristiques à prendre en compte, l'utilisation de métrique pour évaluer le modèle ainsi que la préparation d'un script de prédiction.

Préparation des données

L'objectif du client est de prédire le type de bateau, la variable cible est donc « VesselType ». Il est donc nécessaire de sélectionner les colonnes pertinentes dans la base de données pour prédire. Pour cela, nous avons supprimé les colonnes jugées non pertinentes. Nous avons d'abord supprimé toutes les variables qui étaient uniques à chaque bateau, tel que son Id, VesselName, BaseDateTime, MMSI, IMO et CallSign. Il est aussi essentiel de supprimer VesselType des données car sinon il risquerait de se prédire lui-même et la précision serait de 100%. Les colonnes LAT et LON ont aussi été retirées car cela n'est pas pertinent pour prédire le type lorsque le navire est immobile.



Nous avons étudié la corrélation et l'importance des variables restantes avec VesselType ce qui faciliterait excessivement la prédiction. Nous avons alors exclu Cargo qui a une importance trop importance comme on peut le voir sur le graphique de "l'importance des variables pour la prédiction du type de navire" et COG qui a une corrélation trop faible d'après le graphique de "Corrélation des variables avec VesselType". De plus, on observe l'importance des variables Heading, Draft, Length, Width, Status et SOG pour la prédiction de VesselType. Ce seront donc les variables à extraire.

Ensuite, nous avons supprimé les lignes contenant des valeurs manquantes, encodé les colonnes catégorielles puis normalisé les données pour améliorer la qualité de l'apprentissage.

Nous avons ensuite réalisé l'apprentissage supervisé. Les données ont été divisées en une base d'apprentissage (80 %) et une base de test (20 %). De plus, nous avons appliqué la technique SMOTE afin de rééquilibrer les classes, car les données sont très déséquilibrées, comme le montre l'annexe, avec une prédominance des types 60, 70 et 80.

Apprentissage supervisé pour la classification

VesselType est une variable qualitative, donc pour la prédire, il faut appliquer un modèle d'apprentissage supervisé pour la classification. Pour déterminer quel est le meilleur modèle, nous allons tester deux modèles différents : Random Forest et Régression Logistique. Pour chaque modèle, nous les entraînons avec notre base de données.

Random Forest : C'est une forêt d'arbres, composée de nombreux arbres de décision qui travaillent ensemble afin d'améliorer la précision de la prédiction. Chaque arbre vote pour une classe, puis la classe majoritaire est retenue. Ce modèle permet de réduire le risque de surapprentissage et est robuste et précis surtout pour des bases de données déséquilibrées.

Régression Logistique : Modèle statistique qui estime la probabilité qu'une observation appartienne à une classe en utilisant une fonction logistique appliquée à une combinaison linéaire des variables.

Métriques pour la classification

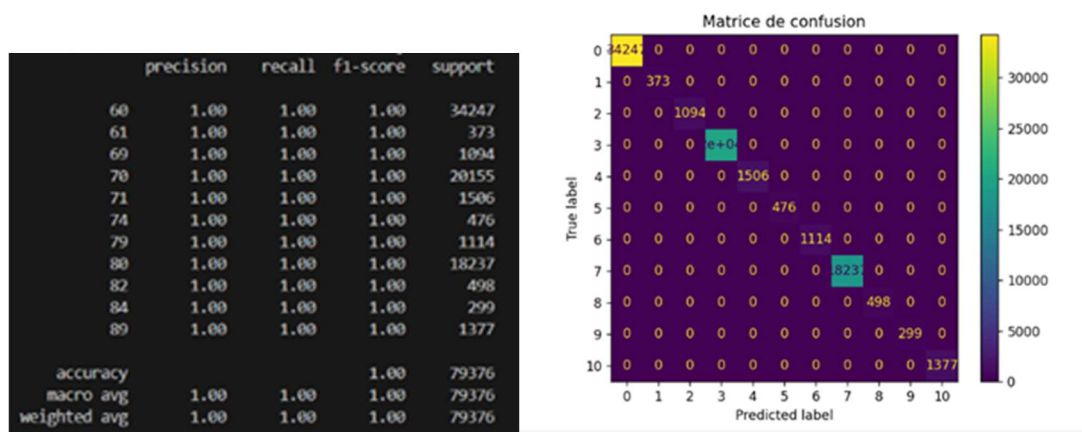
Après avoir nettoyé la base de données puis supprimé les doublons de MMSI, nous avons comparé les performances des modèles RandomForest et Régression Logistique. Pour cela, nous avons utilisé la métrique accuracy, qui permet de mesurer le taux d'exactitude des prédictions.

Les résultats obtenus sont les suivants :

- **RandomForest** : 0,6552
- **Régression Logistique** : 0,517

Ces résultats montrent que RandomForest a une meilleure exactitude que la Régression Logistique. Cela s'explique par sa structure basée sur un ensemble d'arbres de décision : RandomForest est plus adapté à une base de données hétérogène, c'est donc le modèle que nous avons choisi pour continuer notre étude.

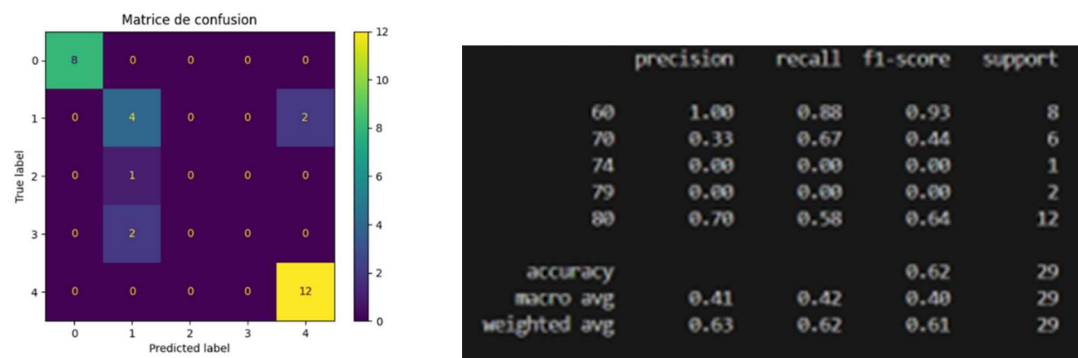
Nous avons commencé nos tests avec RandomForest et avons constaté une précision de 100%, ce qui semblait suspect. La matrice de confusion était également parfaite. Cela s'explique par la grande simplicité de la base de données. Cela s'explique par la simplicité excessive de la base de données : ce sont très souvent les mêmes types de bateaux (60, 70, 80), souvent avec les mêmes caractéristiques. Le modèle d'apprentissage supervisé parvient donc très facilement à prédire ces types de bateau.



Dans ce contexte, le modèle apprend et teste sur les mêmes navires, ce qui rend la prédiction très facile. Comme on peut le voir dans l'annexe 2, de nombreux enregistrements possèdent le même identifiant MMSI, ce qui signifie que les mêmes bateaux sont répétés plusieurs fois dans les données. Le modèle s'entraîne donc sur les mêmes données qu'il est censé prédire.

Pour vérifier cela, nous avons analysé la répartition des MMSI et constaté la présence massive de doublons. Nous avons alors pris la décision de supprimer tous les doublons de MMSI, réduisant la taille de notre jeu de données de 396 878 à 141 navires uniques.

Après cette suppression, la précision du modèle a chuté à 0,62, ce qui est bien plus réaliste. La matrice de confusion n'était plus parfaitement alignée sur la diagonale, signe que le modèle commettait des erreurs mais c'est souhaitable, car cela reflète une meilleure généralisation.



Afin d'améliorer la précision de notre modèle, nous avons cherché à optimiser ses hyperparamètres à l'aide de GridSearchCV. Cette méthode permet de tester différentes combinaisons de paramètres afin de déterminer celles qui offrent les meilleures performances. Grâce à GridSearchCV, nous avons ajusté plusieurs hyperparamètres tels que `n_estimators`, `max_depth`, `min_samples_split` et `min_samples_leaf`.

```
Meilleur score (cross-validation) : 0.6426877470355732
Meilleurs paramètres : ('max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 100)
Meilleur modèle : RandomForest à sauvegarder.
Accuracy : 0.6552
```

Avant cette optimisation, l'accuracy du modèle était inférieure, mais grâce à ce réglage, elle a augmenté pour atteindre 0,66, témoignant ainsi d'un gain de performance significatif et d'un meilleur ajustement du modèle à la base de données.

Script

Le script final fonctionne en ligne de commande et permet à l'utilisateur d'entrer les valeurs de Heading, Draft, SOG, Width, Length et Status d'un navire. Ces données sont ensuite transmises au modèle préentraîné (`model.pkl`). En sortie, le script affiche la catégorie générale du navire ainsi que son type exact prédit par le modèle.

```
python3 .\script.py --heading 90 --draft 3.5 --length 80 --width 12 --sog 5 --status 1
```

```
Catégorie prédit : Cargo
Type prédit : 70
Détail exact : Cargo, all ships of this type
```

Besoin client 3 : Prédiction de la trajectoire des navires

Le troisième besoin du client porte sur la prédiction de la trajectoire future des navires à différents horizons temporels. À partir des données historiques de position, de vitesse, de cap et d'autres variables pertinentes, il s'agit d'entraîner un modèle capable d'anticiper les coordonnées futures d'un navire. Ce besoin relève de la régression dans un contexte de séries temporelles. Pour effectuer une prédiction, il existe différentes méthodes d'apprentissage. Nous avons choisi de tester 3 méthodes : la régression linéaire, random forest et la régression polynomiale. Les variables utilisées comme entrées sont :

- SOG (Speed Over Ground)
- Sin et cos du COG (Course Over Ground)
- Sin et cos du Heading

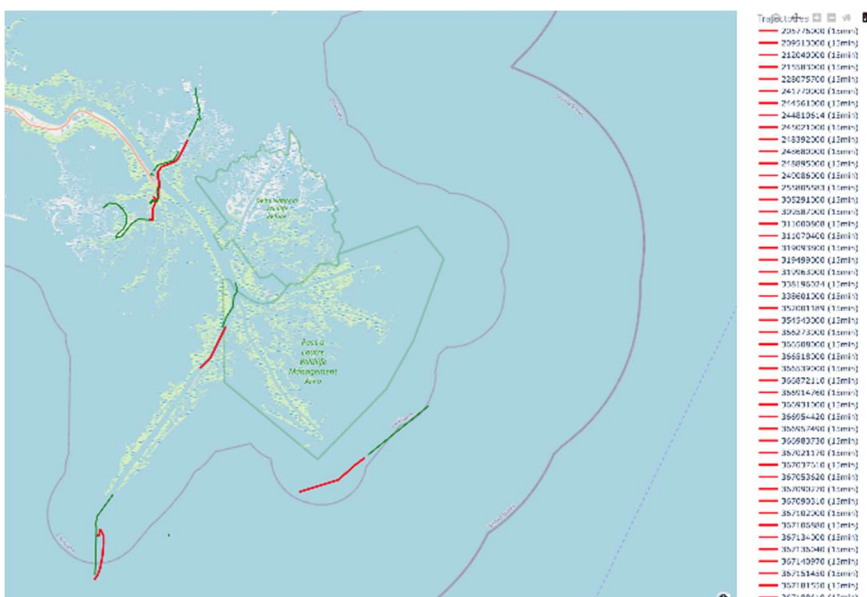
Les sorties prédites sont les différences de latitude et de longitude à un pas de temps futur donné (5min,10min,15min). Enfin, nous pourrions analyser les prédictions avec l'erreur de Haversine (en mètres), le R^2 et le RMSE.

Régression linéaire

La régression linéaire est un modèle simple qui permet d'apprendre une relation linéaire entre les variables d'entrée et les déplacements à venir (delta_LAT, delta_LON). On utilise StandardScaler pour normaliser les données. Ce modèle est très simple et par conséquent plus sujet à des perturbations si les données sont mal nettoyées et si les valeurs d'entrées sont mal choisies.

Random forest

Le modèle Random Forest est un ensemble d'arbres de décision. Il peut capter des relations non linéaires entre les variables et réduit le risque de sur-apprentissage. Nous l'avons entraîné avec 100 arbres ($n_estimators=100$, $random_state=42$). L'apprentissage de ce modèle est efficace que les autres pour des interactions complexes. Mais en contrepartie il est très lourd en mémoire et très long.



Affichage sur carte des trajectoires réel (vert) et prédit (rouge) réalisé avec random forest

Régression polynomiale

La régression polynomiale ajoute des interactions non linéaires entre les variables d'entrée. On applique une régression linéaire, puis on utilise un pipeline scikit-learn pour automatiser la création des variables polynomiales (de degré 2). Ce modèle offre souvent une meilleure précision que la régression linéaire tout en restant simple à interpréter. Plus précis tout en restant rapide, c'est le meilleur compris entre les trois modèles.

Les modèles sont évalués sur des jeux de test séparés pour chaque navire, avec prédiction sur les dernières minutes. On remarque que le modèle le plus pertinent est le Random Forest qui

```
### 15 min ###

--- Régression Linéaire ---
Taille données entraînement : (173678, 15)
Taille données test : (970, 15)
Performance globale : R² = 0.9998, RMSE = 0.066284
Erreur de Haversine moyenne : 894.01 mètres
Erreur de Haversine - écart type : 9219.41 mètres

--- Random Forest ---
Taille données entraînement : (173678, 15)
Taille données test : (970, 15)
Performance globale : R² = 0.9998, RMSE = 0.066777
Erreur de Haversine moyenne : 756.51 mètres
Erreur de Haversine - écart type : 9306.21 mètres

--- Régression Polynomiale ---
Taille données entraînement : (173678, 15)
Taille données test : (970, 15)
Performance globale : R² = 0.9998, RMSE = 0.066119
Erreur de Haversine moyenne : 847.82 mètres
Erreur de Haversine - écart type : 9199.89 mètres
```

possède l'erreur de Haversine moyenne la plus basse. On peut aussi noter que cette erreur est généralement très élevée pour tous les modèles dans toutes les échelles de temps. Cela montre que la prédiction. Après cette analyse, nous avons décidé d'utiliser le modèle random forest pour notre script.

Script

L'utilisateur rentre dans les valeurs d'entrée nécessaire au modèle d'apprentissage. Le script prépare ces valeurs puis utilise les modèles déjà entraînés pour prédire la position d'un seul bateau.

```
--sog 13.4 --cog 131 --latitude 29.19717 --longitude -94.4992 --heading 131 --minutes 15
Modèle chargé: random_forest_15.pkl
Position prédite sur 15 minutes:
LAT: 29.193039
LON: -94.493640
```

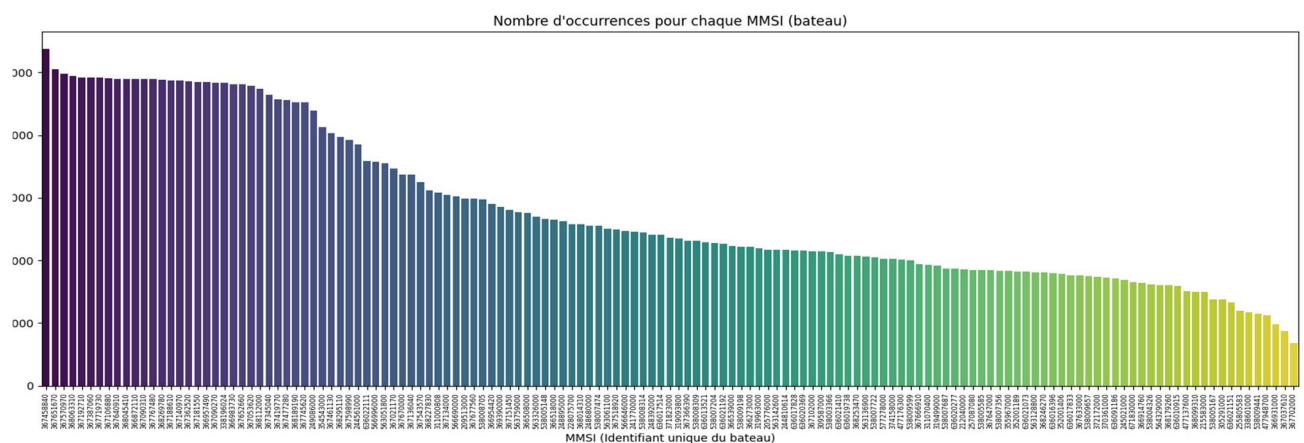
Exemple de sortie de script pour un bateau quelconque

ANNEXE

Annexe 1 :

	precision	recall	f1-score	support
60	1.00	1.00	1.00	34247
61	1.00	1.00	1.00	373
69	1.00	1.00	1.00	1094
70	1.00	1.00	1.00	20155
71	1.00	1.00	1.00	1506
74	1.00	1.00	1.00	476
79	1.00	1.00	1.00	1114
80	1.00	1.00	1.00	18237
82	1.00	1.00	1.00	498
84	1.00	1.00	1.00	299
89	1.00	1.00	1.00	1377
accuracy			1.00	79376
macro avg	1.00	1.00	1.00	79376
weighted avg	1.00	1.00	1.00	79376

Annexe 2 :



Bibliographie

- <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html>